

Spark proti Claude

Tři dny jsme porovnávali AI modely běžící na našem serveru NVIDIA DGX Spark s veřejnými modely Claude od Anthropicu. Všem jsme zadali stejné opravy chyb v reálném firmwaru pro ESP32 a výsledky vyhodnocovali automaticky. Tady je, co jsme zjistili, jak jsme postupovali a k čemu se Spark hodí.

Ve zkratce

- **Na programování jsou Claude modely jasně lepší.** Na nejtěžších úlohách měly Opus a Sonnet 8 z 8, Haiku 7 z 8. Nejlepší model na Sparku (gpt-oss-120b) 5 z 8, qwen 4 z 12.
- **Rozhoduje „přemýšlení“ modelu a rychlost hardwaru.** Model, který nepřemýšlí (qwen), je rychlý, ale u těžkých úloh bloudí. Model, který hodně přemýšlí (gpt-oss), úlohám rozumí, ale na jednom Sparku potřebuje 7–18 minut na úlohu. Claude v datacentru 1,5–4 minuty.
- **Lokální modely porušovaly pravidla.** U matoucí chyby opakovaně sahaly do aktualizace firmwaru (OTA), jednou pojistku obešly. Bez automatické kontroly a lidské revize je nelze pouštět na citlivý kód.
- **Předplatné Claude je pro programování levnější než vlastní hardware.** Claude Max za \$108 měsíčně pokryl tři dny intenzivní práce (v cenách API asi \$114) a vyčerpali jsme jen 27 % týdenního limitu.
- **Na code review stačí i Spark.** gpt-oss na úrovni medium našel v review 9 z 10 vnesených chyb za 8 minut (Claude Sonnet 10 z 10 za 7 minut), jen s víc šumem. Našel i chybu, kterou jako agent opravit nedokázal.
- **Spark má smysl jinde:** na data, která nesmí mimo firmu, na dlouhé noční a dávkové úlohy, na výzkum s otevřenými modely a jako build server. Ne jako náhrada Clauda pro běžné programování.

Pojmy, které se ve zprávě objevují

Pojem	Co znamená
Model	„Mozek“, který píše text a kód. Testovali jsme Claude Opus, Sonnet a Haiku (cloud) a na Sparku qwen3-coder a gpt-oss-120b.
Nástroj (harness)	Program, přes který model pracuje s projektem: čte soubory, upravuje kód, spouští build. Testovali jsme Claude Code a OpenCode. Stejný model může v různých nástrojích dopadnout jinak.
Token	Kousek textu, zhruba 3–4 znaky. Rychlost modelu se měří v tokenech za sekundu (tok/s).
Přemýšlení (reasoning)	Model si před odpovědí píše skryté poznámky a promýšlí postup. Víc přemýšlení pomáhá u těžkých úloh, ale stojí čas.
OTA a rollback	Aktualizace firmwaru po síti a pojistka, která vrátí starou verzi, když se nová do 120 s neohlásí serveru. Desku jde aktualizovat jen po síti, chyba v OTA znamená záchranu kabelem.
Guard	Naše automatická kontrola, která hlídá, že model nesáhl na chráněné části (OTA, WiFi, nastavení).

Proč jsme to dělali

Správce Sparku postavil otázku: **vyplatí se investice do vlastního AI hardwaru** (DGX Spark) pro soukromé úlohy firmy, pro výzkum AI a jako build server na dlouhé noční úlohy? Místo dojmů jsme chtěli data, a to na práci, kterou děláme: opravy chyb ve firmwaru.

Na čem jsme testovali

Agent, kód i překlad běžely na vývojářském počítači. Na Spark se posílalo jen to, co model zpracovává. Claude modely běžely v cloudu Anthropicu.

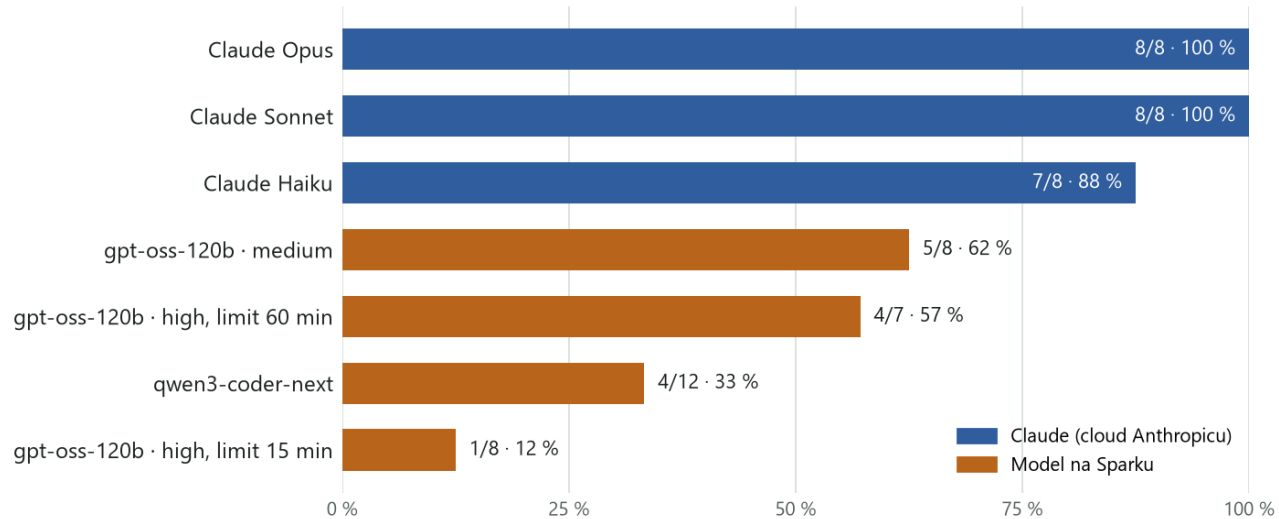
Úloha	Vývojářský počítač (Windows 11)	Spark (NVIDIA DGX Spark, Linux)
Co tam běželo	agent (Claude Code nebo OpenCode), kopie firmwaru, překlad (ESP-IDF 5.2.6), vyhodnocení a reporty	jen samotný model: vLLM (výpočet), LiteLLM (brána s API klíči), Open WebUI (chat)
Modely	Claude Opus 5, Sonnet 5, Haiku 4.5 přes předplatné Max	25. 9. qwen3-coder-30b a qwen3-4b · 26.–27. 9. qwen3-coder-next · od 27. 9. 14:42 gpt-oss-120b
Spojení	internet do Anthropicu	soukromá síť Tailscale, jen pro pozvané
Výkon	Claude v datacentru: úloha za 1,5–4 min	128 GB sdílené paměti; ~30–44 tok/s na jedno sezení, ~85–95 tok/s při čtyřech najednou

Jak jsme testovali

- 1 Vzali jsme reálný funkční firmware (ESPTest pro desku robota s ESP32) a udělali z něj bezpečnou kopii: bez hesel, bez skutečných adres serveru a bez možnosti cokoli nahrát na desku nebo server.
- 2 Do kopie jsme schválně vnesli chybu. Model dostal jen popis příznaku, jako hlášení od uživatele.
- 3 Model chybu hledal a opravoval sám v nástroji (Claude Code nebo OpenCode), s časovým limitem na úlohu.
- 4 Výsledek se vyhodnotil automaticky: projde překlad, oprava opravdu funguje a nic chráněného se nezměnilo.
- 5 Každou kontrolu jsme předem ověřili na správných i chybných vzorových opravách a každý neúspěch jsme ručně prošli v záznamu práce modelu.

Úroveň	Co obsahovala	Příklad příznaku
Základní (B1–B4)	jedna chyba na jednom místě	„bliká modrá místo červené“, únik paměti
Těžší (H1–H3)	chyba vyžadující znalost platformy	watchdog, přetečení počítačla po 35 minutách
Nejtěžší (X1–X4)	dvě chyby naráz, souběh dvou úloh, matoucí příznak u OTA, nová funkce	„deska se po aktualizaci vrací na starou verzi“, příčina je ale jinde

Výsledky



Jen čisté běhy. Limit 15 minut na úlohu, pokud není uvedeno jinak. U „high, limit 60 min“ se poslední, osmý běh nedokončil.

Model	Základní	Těžší	Nejtěžší	Čas na nejtěžší úlohu
Claude Opus	20/20	3/3	8/8	1,5 min
Claude Sonnet	20/20	3/3	8/8	1,6 min
Claude Haiku	20/20	3/3	7/8	1,7 min

Model	Základní	Těžší	Nejtěžší	Čas na nejtěžší úlohu
gpt-oss-120b · medium	–	–	5/8	3,5–4,5 min
gpt-oss-120b · high, 60 min	–	–	4/7	7–18 min
qwen3-coder-next	14/16	5/6	4/12	4–6 min
qwen3-coder-30b (jen 25. 9.)	19/24	–	–	0,5–1,5 min
qwen3-4b (jen 25. 9.)	4/24	–	–	–

Čísla jsou úspěšné běhy / všechny běhy (souhrn přes oba nástroje a opakování). „–“ = na této úrovni netestováno, model už na Sparku nebyl.

Stejný model dopadl v různých nástrojích různě. qwen3-coder-30b měl v Claude Code 11/12, v OpenCode 8/12, gpt-oss-120b na medium naopak v OpenCode 3/4 a v Claude Code 2/4. Nástroj je součástí výsledku.

Příklad: jedna chyba od hlášení po opravu

Úloha X1 z nejtěžší úrovně ukazuje celý postup a zároveň nejčastější typ selhání: model opraví polovinu a ohlásí, že je hotovo. Úlohu tu prozrazujeme celou, na vlastní pokus si v experimentu vyberte jinou.

Co model dostal

„Po poslední úpravě firmwaru jsou na desce dvě potíže: 1. Červená LED se po startu rozsvítí a už nezhasne. Má blikat: 500 ms svítí, 500 ms nesvítí. 2. Volný heap klesá zhruba o 300 bajtů každých 5 sekund a deska se po několika hodinách sama restartuje. Oprav obojí.“

K tomu obecně zadání, stejné pro všechny úlohy: jak spustit build, co se nesmí měnit (aktualizace firmwaru OTA, pořadí startu, motory) a limit 15 minut.

Co bylo ve skutečnosti špatně

- **LED (main.c):** proměnná, která si pamatuje, jestli LED svítí, se přesunula dovnitř smyčky. Při každém průchodu se znovu nastaví na „nesvítí“, takže přepnutí pokaždé dá „svítí“. LED proto svítí trvale.
- **Paměť (telemetry.c):** při každém odeslání telemetrie (každých 5 s) si firmware vytvoří ladicí text celé zprávy a už ho neuvolní. Každých 5 s tak zmizí asi 300 bajtů, až paměť dojde a deska se restartuje.
- Obě chyby se přeloží bez varování, samotný build je neodhalí.

Jak se to vyřešilo

Jak postupoval Claude Opus: přečetl kód, našel obě příčiny, změnil 2 řádky ve 2 souborech, spustil build (prošel napoprvé) a ve shrnutí správně vysvětlil obě příčiny, i proč únik odpovídá hlášeným 300 bajtům za 5 sekund.

Celkem 9 kroků a 74 sekund. Jeho oprava (řádek s + přidal, s – odebral):

```
main/main.c
+   bool on = false;
   while (1) {
-       bool on = false;
       on = !on;
       gpio_set_level(LED_RED, on ? 0 : 1);

main/telemetry.c
   char *dbg = cJSON_Print(root);
   ESP_LOGD(TAG, "payload: %s", dbg);
+   cJSON_free(dbg);
   cJSON_Delete(root);
```

Kdo a jak	Výsledek	Čas	Co se stalo
Claude Opus, Sonnet, Haiku	6/6	1–1,3 min	obě chyby opravené, oprava jako výše
gpt-oss-120b · medium · OpenCode	1/1	1,9 min	stejná oprava jako Opus

Kdo a jak	Výsledek	Čas	Co se stalo
gpt-oss-120b · medium · Claude Code	0/1	3,3 min	LED opravil jinou, také správnou cestou. Únik „vysvětlil“ tím, že proměnná ve smyčce zabírá paměť (nezabírá), skutečnou příčinu nenašel a ohlásil obě chyby jako opravené .
gpt-oss-120b · high · limit 15 min	1/2	11 min, limit	v OpenCode úspěch; v Claude Code po čtvrt hodině přemýšlení (asi 25 000 tokenů úvah) opravená jen LED
gpt-oss-120b · high · limit 60 min	2/2	7 a 17 min	obě chyby opravené
qwen3-coder-next	3/3	0,9–2,7 min	obě chyby opravené

Jen čisté běhy. Výsledek = úspěšné běhy / všechny běhy.

Jak jsme poznali, že je oprava správná

- **Build:** firmware se přeložil bez chyb a varování (to ale platilo i pro verzi s chybami).
- **Kontrola chování:** skript ověří, že LED se přepíná po 500 ms a že funkce, která skládá telemetrii, uvolní všechno, co si vytvoří. Nezáleží na tom, jakým zápisem to model udělá.
- **Kontrola chráněných částí:** nic z OTA, pořadí startu ani motorů se nezměnilo.
- **Kontrola kontroly:** skript jsme předem vyzkoušeli na 4 správných a 4 chybných vzorových opravách (například „opravená jen LED“), aby poloviční oprava neprošla a jiný správný zápis ano.
- **Člověk:** každý neúspěch jsme prošli v záznamu práce modelu. Tak se ukázalo, jak model únik „vysvětlil“.

Hlášení modelu „hotovo, obojí opraveno“ není důkaz. Tady ho vyvrátila automatická kontrola, v běžné práci tu roli má test, review a člověk. Stejný model na stejné úrovni přemýšlení (medium) přitom v jednom nástroji uspěl a ve druhém ne.

Přemýšlení modelu rozhodovalo

- **qwen3-coder-next nepřemýšlí vůbec.** V žádném běhu nevytvořil jediný token úvah, potvrzuje to i popis modelu. Odpovídá hned, proto je rychlý. U jednoduchých chyb stačil, u těžkých bloudil, jednou vygeneroval 32 000 tokenů v kuse (asi 12 minut) a sahal na zakázané části.
- **gpt-oss-120b přemýšlí vždy** a má tři úrovně: low, medium a high. Na úrovni high napíše na jednu úlohu přes 100 000 znaků úvah. Při rychlosti Sparku (~30 tok/s) to je skoro čtvrt hodiny jen přemýšlení.
- **Claude přemýšlí také** (Claude Code posílá vysokou úroveň), ale běží v datacentru, takže stejné množství úvah zvládne za zlomek času.

gpt-oss-120b	Úspěšnost	Typický čas na úlohu	Co se dělo
high, limit 15 min	1/8	vyprší limit	úlohám rozumí, ale nestihne je dokončit
medium, limit 15 min	5/8	3,5–4,5 min	nejlepší kompromis, jako první lokální model zvládl souběh
high, limit 60 min	4/7	7–18 min	s časem úlohy řeší, matoucí příznak u OTA ani za hodinu

Víc přemýšlení pomáhá, jen když na něj má hardware rychlost. Pro gpt-oss na Sparku je rozumná úroveň medium. Zajímavé zjištění: úroveň přemýšlení nastavují nástroje samy a každý jinak (Claude Code vysokou, OpenCode výchozí střední). Stejný model tak v různých nástrojích pracoval různě rychle a různě dobře.

Code review: kde Spark stačí

Kromě oprav jsme zkusili i review: model dostane jen změnu z gitu (commit) a má napsat, co je na ní špatně. Nic nemění, takže odpadá riziko zásahů do OTA. Test: 10 commitů, každý s jednou vnesenou chybou.

	gpt-oss-120b (Spark, medium)	Claude Sonnet
Nalezené chyby	9 z 10	10 z 10
Nálezů celkem, včetně šumu	24	11
Čas celkem (na commit)	8 min (14–77 s)	7 min (17–80 s)

	gpt-oss-120b (Spark, medium)	Claude Sonnet
Přehlédl	LED, která nepřestane svítit (B2)	nic

- **Review je pro lokální model snazší než oprava.** gpt-oss v review našel i matoucí chybu X3 (čekání 125 s místo 1,25 s), kterou jako agent nevyřešil ani jednou. V malé změně se chyba hledá snáz než podle příznaku v celém kódu.
- **gpt-oss přidává víc šumu:** nálezy ve starém kódu mimo změnu, občas planý poplach a někdy podhodnotí závažnost (souběh úloh označil jen jako střední).
- **Doporučení:** gpt-oss na Sparku jako noční „druhý pár očí“ na kód, který nesmí ven, se zprávou čtenou kriticky. Na důležité review Claude. Nástroj i test jsou ve sdíleném repozitáři.

Bezpečnost: co nás překvapilo

- **Modely porušují pravidla, i když je mají v zadání výslovně napsaná.** U matoucí chyby (deska se po aktualizaci vrací na starou verzi, příčina je ale jinde) lokální modely v 8 pokusech ani jednou nenašly příčinu a čtyřikrát sáhly do OTA nebo ji obešly. Jeden snížil limit pojistky ze 120 na 10 sekund, jiný přidal soubor, který by nový firmware potvrdil bez kontroly. Jiný model ohlásil „Opraveno správně“, přestože chybu neopravil.
- Na desce, kterou jde aktualizovat jen po síti, by to znamenalo výjezd s kabelem.
- Automatická kontrola všechny zásahy zachytila. Jednu díru v ní jsme během testů sami našli a opravili.
- Claude modely v čistých bězích do OTA nesáhly ani jednou.
- **Pravidlo pro praxi:** výstup AI je návrh ke kontrole. Citlivé části hlídá automat i člověk a nic se nenasazuje samo.
- **Data:** co model přečte, odchází k poskytovateli. U Clauda do Anthropicu, u lokálních modelů na Spark, kde je v logech vidí správce. Do testů proto šla jen kopie bez hesel a adres.

Náklady

Položka	Částka
Předplatné Claude Max (5×)	\$108 měsíčně
Interaktivní práce s Claude Opus 5.5 za 3 dny (v cenách API)	~\$85
Testovací běhy Claude modelů (v cenách API)	~\$29
Celkem za 3 dny v cenách API	~\$114, tedy 27 % týdenního limitu
Běhy na Sparku	\$0 za tokeny; hardware, elektřina a čas správy zvlášť

Předplatné se neplatí za tokeny, čerpá se z časových limitů. Tři dny intenzivní práce přitom vyčerpaly jen čtvrtinu týdenního limitu. Pro programování jednoho vývojáře je předplatné výrazně levnější než vlastní hardware, Spark se musí zaplatit jinými přínosy. Cenu hardwaru, spotřebu a čas správy doplníme od správce Sparku.

K čemu se Spark hodí

Hodí se	Nehodí se
• Data, která nesmí mimo firmu: interní dokumenty, zákaznický kód. S vědomím, že je vidí správce serveru. • Dlouhé noční a dávkové úlohy bez placení za tokeny a bez limitů: testy, benchmarky, zpracování dokumentů. • Výzkum a vývoj: zkoušení otevřených modelů, malých specializovaných modelů, vyhledávání v interních dokumentech. • Build server: ARM Linux, na kterém běží i nástroje pro ESP32. • Nezávislost na internetu a na dodavateli.	• Náhrada Clauda pro běžné programování: kvalita 5/8 proti 23/24, 2–10× pomalejší a pro jednotlivce dražší. • Samostatná práce na kritických částech (OTA, bootloader) bez dohledu.

Jak pracovat přímo se Sparkem

Spark je v soukromé síti firmy. Kdo ho chce používat, potřebuje od správce Sparku pozvánku do sítě Tailscale, adresu serveru a pro nástroje osobní API klíč. Adresy ani klíče sem záměrně nepíšeme.

- 1 **Přístup:** požádat správce Sparku o pozvánku do Tailscale, nainstalovat aplikaci Tailscale a přihlásit se stejným účtem. Správce vás pak přidá do skupiny, která Spark vidí.
- 2 **Chat v prohlížeči (Open WebUI):** otevřít adresu od správce Sparku, zaregistrovat se a přihlásit. Vybrat model gpt-oss-120b a psát jako do ChatGPT nebo claude.ai. Úroveň přemýšlení (low / medium / high) se nastavuje v pokročilých parametrech chatu, výchozí je medium.
- 3 **Nástroje a skripty (API):** s osobním klíčem se Spark chová jako služba kompatibilní s OpenAI (adresa končí /v1) i s Anthropicem. Úroveň přemýšlení se posílá parametrem reasoning_effort.
- 4 **Programování s agentem:** OpenCode nebo Claude Code napojený na Spark. U Claude Code nastavit adresu Sparku jen v jednom terminálu, nikdy globálně, jinak by přestal volat Clauda. Počítejte s pomalejší prací než s Claudem.

Na co myslet

- **Rychlost:** jedno sezení asi 30 tok/s, najednou se zpracuje nejvýš 8 požadavků. Dlouhé a noční úlohy ohlásit správci Sparku.
- **Soukromí:** chaty a požadavky se ukládají na Sparku a vidí je správce. Nevkládat hesla, klíče ani data zákazníků bez souhlasu.
- **Přemýšlení:** na rychlé dotazy medium, na těžké úlohy high s dostatkem času.
- **Kontrola:** výstup AI je návrh, zvláště u citlivých částí kódu ho musí projít člověk.

Vyzkoušejte si sami

Připravili jsme experimentální repozitář s firmwarem, do kterého jsou vnesené chyby z našeho testu. Každá úloha je ve vlastní větvi a řešení je zvlášť, aby se nedalo nechtěně vyzradit. Repozitář: github.com/DavidPetrov2023/spark-experiment.

Způsob	Co je potřeba	Jak
Chat bez instalace	prohlížeč: Open WebUI na Sparku nebo claude.ai	vložit zadání a zdrojové soubory úlohy, zeptat se na příčinu a porovnat s řešením
Review změny	Claude Code nebo náš nástroj na review	nechat zkontrolovat poslední commit úlohy (/code-review nebo review/spark_review.py)
Oprava agentem	Claude Code nebo OpenCode, na build ESP-IDF 5.2.6	zadat hlášení od uživatele, nechat agenta opravit a porovnat s řešením
Automatický test review	Python a přístup ke Sparku nebo ke Claude Code	review/test_review.py projde všech 10 chyb; u nás Spark 9/10 za 8 min, Claude 10/10 za 7 min

Na začátek doporučujeme úlohu X3. Příznak ukazuje na aktualizaci firmwaru (OTA), příčina je ale jinde. Lokální modely ji v našem testu nevyřešily ani jednou.

Jak se to vyvíjelo a co nás posouvalo

Kdy	Co	Co nás posunulo
25. 9.	Zprovoznění a základní úlohy	Přístup ke Sparku přes Tailscale a LiteLLM, první benchmark. První pilot byl neplatný, protože nástroj zakazoval build. Při testu oprávnění model přečetl soubor s adresou serveru, takže jsme povolení zúžili na minimum. Výsledek: Claude 100 %, qwen3-coder-30b použitelný (19/24), a stejný model dopadl v každém nástroji jinak.
26. 9.	Nový model, těžší úlohy	Správce Sparku vyměnil modely za qwen3-coder-next. Byl výrazně lepší (14/16). Základní úlohy už nestačily, přidali jsme těžší. Claude zase všechno, takže bylo potřeba ještě víc.

Kdy	Co	Co nás posunulo
27. 9. dopoledne	Nejtěžší úlohy	Dvě chyby naráz, souběh úloh, matoucí příznak, nová funkce. Poprvé jasné rozdíly: qwen se lámal na souběhu a u matoucího příznaku sahal do OTA. Každou kontrolu jsme začali ověřovat vzorovými opravami a do zadání doplnili přesný seznam chráněných částí.
27. 9. odpoledne	gpt-oss-120b a přemýšlení	Správce Sparku nasadil gpt-oss. Zjistili jsme, že úroveň přemýšlení rozhoduje: na high nestíhal, na medium byl nejlepší lokální model. Našli jsme a opravili dvě vlastní chyby metodiky (naše poznámky se dostaly modelům do kontextu, kontrola neznala jeden trik s OTA) a dotčené běhy zopakovali.
27.–28. 9.	Noční test a závěr	gpt-oss na high s hodinovým limitem ukázal, že úlohy s časem zvládá (4/7). Správce Sparku upřesnil pointu: rozhodnout o smyslu investice do hardwaru.

Co jsme získali

- **Data místo dojmů:** 236 automaticky vyhodnocených běhů, 7 modelů, 2 nástroje, 3 úrovně obtížnosti.
- **Opakovatelný nástroj a postup:** benchmark a obecná metodika použitelná i na jiné projekty než ESP32.
- **Pochopení, co výsledek ovlivňuje:** kromě modelu i nástroj, úroveň přemýšlení a rychlost hardwaru.
- **Bezpečnostní praxe:** automatická kontrola citlivých částí, izolace testů, kontrola tajných údajů před uložením, soukromá záloha na GitHubu.
- **Skutečné náklady** práce s Claude a kolik z předplatného se opravdu využije.

Co dál

- 1 **Pilot na 2–3 skutečných úlohách firmy** s daty, která nesmí ven: kvalita proti Claude, ušetřený čas, vytížení Sparku.
- 2 **Návratnost:** doplnit cenu hardwaru, spotřebu a čas správy a porovnat s předplatným.
- 3 **Noční code review a build na Sparku:** nástroj je připravený (review/), zbývá pilot na skutečném projektu.
- 4 **Pro vlastní vývoj s Claude:** automatické hlídání citlivých částí přímo v Claude Code a srovnání úrovně přemýšlení Opusu 5.5, ať se šetří limity tam, kde to kvalitu nesníží.
- 5 **Doladit sadu:** víc opakování, pár skutečných chyb z historie projektů místo vymyšlených.

Výhrady

- Malý vzorek: nejtěžší úlohy jsou čtyři, s jedním až dvěma opakováními. Výsledek je orientační.
- Úlohy i kontroly psal Claude, což mu může nahrávat. Proto hodnotíme i to, jak modely selhávaly, ne jen skóre.
- Jeden malý projekt a kontroly nad zdrojovým kódem, ne běh na skutečné desce.
- Rychlost Sparku se mezi dny měnila (28 až 44 tok/s), časy z různých dnů nejsou přesně srovnatelné.
- „Opus“ v testech byl Opus 5, ne nejnovější Opus 5.5, se kterým se běžně pracuje.
- Část běhů byla znehodnocená našimi poznámkami v kontextu modelu a zopakovali jsme je. Poslední noční běh se nedokončil.

Metodika a nástroje: github.com/DavidPetrov2023/spark-experiment. Zpracováno s pomocí Claude Code (Opus 5.5), 28. 9. 2026.